Desenvolvedor Web Back-end: Node.js

Introdução

Desenvolvedor Web Back-end: Node.js

Duração: 40 horas

Início: 28/03/2026

Previsão de Termino: 18/06/2026

Sábados das 8:00h às 12:00h

Apresentação Pessoal

Expectativa…

Google Drive

https://forms.gle/HG2UycU3KpYRYdT76

GitHub e Google Drive

Repositórios do curso...

GitHub

https://github.com/leonardoignacio/nodejs2026

Google Drive

https://drive.google.com/drive/folders/15CBwSYYTceXT0yAiHkbBAExU7d2BCSuk?usp=drive_link

WhatsApp…

Gupo WhatsApp

https://chat.whatsapp.com/LQdWqksa7XiEGSTQb8T4g8?mode=gi_t

Sobre o Curso

Título: Desenvolvedor Web Back-end: Node.js

Modalidade: Aperfeiçoamento

Carga Horária: 40 horas

Objetivo

Desenvolver aplicações back-end.

Para **profissionais e estudantes** que desejam **adquirir e/ou ampliar** seus conhecimentos em projetos web tendo em vista o negócio próprio ou a atuação em diversos contextos profissionais tais como: webdesigners, **desenvolvedores back-end**, Programadores **Web** ou **mobile**, entre outros.

Desenvolvedor Web Back-end: Node.js

Unidades curriculares	Carga horária
UC1: Desenvolver aplicações back-end.	40 horas
Carga horária total	40 horas

Objetivo de aprendizagem

- Reconhecer o ambiente e fundamentos do Node.js
- Ampliar o repertório técnico da linguagem javaScript para back-end.
- Reconhecer as ferramentas e frameworks para desenvolvimento back-end.
- Desenvolver aplicação back-end que manipula banco de dados.
- Reconhecer arquiteturas e padrões de desenvolvimento Web.
- Utilizar o HTML engine para renderizar páginas dinâmicas.
- Ampliar o repertório técnico na integração em back-end e front-end.

Sobre o Curso

UC1: Desenvolver aplicações back-end

CARGA HORÁRIA: **40 horas**

Indicadores

1. Configura o ambiente de desenvolvimento back-end conforme os padrões do Node.js.
2. Cria serviços web, conforme padrões definidos pela arquitetura REST.
3. Manipula banco de dados conforme requisitos do projeto.

Elementos da competência

CONHECIMENTOS

- Ambiente de desenvolvimento: instalação e configuração de ambientes locais.
- Conceitos da runtime Node.js.
- Gerenciador de pacotes NPM, módulos internos, externos e core modules.
- Banco de dados: Conceito de MER; Banco de dados MySQL e instruções SQL; Conceito de ORM e Models.
- Linguagem JavaScript para back-end: funções, callbacks e middleware; programação síncrona e assíncrona (Promises e Async/Awat).
- Integração com o front-end: Axios (cliente HTTP) e EJS (html engine).
- Padrão de arquitetura MVC: estrutura da aplicação.
- API: Arquitetura Restful; protocolo HTTP e framework Express.
- Integração com o front-end: Axios (cliente HTTP) e EJS (html engine).

HABILIDADES

- Interpretar briefing.
- Utilizar linguagem de programação JavaScript.
- Utilizar conceitos de programação orientada à objetos.
- Selecionar e interpretar requisitos de usuário.
- Utilizar boas práticas de programação e padrões de projetos.
- Utilizar termos técnicos nas rotinas de trabalho.

Estrutura do curso

Situação de aprendizagem

1. Ambiente e fundamentos do Node.js.

Indicadores

1. Configura o ambiente de desenvolvimento back-end conforme os padrões do Node.js.
2. Cria serviços web, conforme padrões definidos pela arquitetura REST.

Elementos da competência

CONHECIMENTOS

- Ambiente de desenvolvimento: instalação e configuração de ambientes locais.
- Conceitos da runtime Node.js.
- Gerenciador de pacotes NPM, módulos internos, externos e core modules.
- API: Arquitetura Restful, Protocolo HTTP, framework Express.

HABILIDADES

- Interpretar briefing.
- Utilizar linguagem de programação JavaScript.
- Utilizar conceitos de programação orientada à objetos.
- Utilizar termos técnicos nas rotinas de trabalho.

Problematização

Um desenvolvedor front-end foi contratado por uma empresa de pet shop para criar um espaço que facilite a adoção de pets abandonados. Para este trabalho serão exigidos conhecimentos e domínio de alguns fazeres que vão além do front-end.

Questões norteadoras:

- O que é necessário para o desenvolvimento do sistema?
- O que é back-end? Quais tecnologias são necessárias?
- Como facilitar a manutenção e expansão das funcionalidades?
- Como integrar o back-end com o a interface do usuário?
- O que é uma API?
- O que é requisição e o que são verbos HTTP?

Estrutura do curso

Situação de aprendizagem

2. Modelo de Entidade e Relacionamento, banco de dados e integração com a API.

Indicadores

2. Cria serviços web, conforme padrões definidos pela arquitetura REST.
3. Manipula banco de dados conforme requisitos do projeto.

Elementos da competência

CONHECIMENTOS

- Banco de dados: Conceito de MER; Banco de dados MySQL e instruções SQL; Conceito de ORM e Models.
- API: Arquitetura Restful, protocolo HTTP e framework Express.

HABILIDADES

- Interpretar briefing.
- Utilizar linguagem de programação JavaScript.
- Utilizar conceitos de programação orientada à objetos.
- Selecionar e interpretar requisitos de usuário.
- Utilizar boas práticas de programação e padrões de projetos.
- Utilizar termos técnicos nas rotinas de trabalho.

Problematização

Após conhecer tecnologias back-end e escolher o Node.js para desenvolver o sistema, precisa pensar como manter, organizar e acessar os dados:

Questões norteadoras:

- ✓ Como as informações são armazenadas?
- ✓ O que é um banco de dados? Como projetá-lo?
- ✓ Quais tecnologias de bancos de dados existem?
- ✓ Qual SGBD utilizar? Como implementar um banco de dados para a aplicação?
- ✓ Como conectar a API ao banco de dados?
- ✓ Como inserir, ler, atualizar e excluir registros por meio da aplicação?

Estrutura do curso

Situação de aprendizagem

3. Organização do código, conceito ORM e o acesso ao banco utilizando models.

Indicadores

2. Cria serviços web, conforme padrões definidos pela arquitetura REST.
3. Manipula banco de dados conforme requisitos do projeto.

Elementos da competência

CONHECIMENTOS

- Padrão de arquitetura MVC: estrutura da aplicação.
- Banco de dados: Conceito de MER; Banco de dados MySQL e instruções SQL; Conceito de ORM e Models.
- API: Arquitetura Restful, Protocolo HTTP, framework Express.
- Integração com o front-end: Axios (cliente HTTP) e EJS (html engine).

HABILIDADES

- Interpretar briefing.
- Utilizar linguagem de programação JavaScript.
- Utilizar conceitos de programação orientada à objetos.
- Selecionar e interpretar requisitos de usuário.
- Utilizar boas práticas de programação e padrões de projetos.
- Utilizar termos técnicos nas rotinas de trabalho.

Problematização

Após criar uma estrutura de banco de dados desenvolver a aplicação é necessário organizar o código para facilitar a manutenção e atender novas necessidades futuras. Também é necessário cuidar da segurança dos dados criando métodos de acesso que não apresente vulnerabilidades a ataques.

Perguntas norteadoras:

- ✓ Como organizar os arquivos e o código de uma aplicação de forma lógica?
- ✓ O que é um padrão de projeto (design patter)?
- ✓ Como estruturar a aplicação de acordo com o padrão MVC?
- ✓ Existem outras formas para a aplicação manipular o banco de dados?
- ✓ O que é um ORM e quais problemas ele resolve?
- ✓ Como implementar um models em uma aplicação Node.js?

Estrutura do curso

Situação de aprendizagem

4. Interface da aplicação, renderização de templates dinâmicos e consumo de serviços do back-end.

Indicadores

2. Cria serviços web, conforme padrões definidos pela arquitetura REST.

Elementos da competência

CONHECIMENTOS

- Banco de dados: Conceito de MER; Banco de dados MySQL e instruções SQL; Conceito de ORM e Models.
- Padrão de arquitetura MVC: estrutura da aplicação.
- Integração com o front-end: Axios (cliente HTTP) e EJS (html engine).

HABILIDADES

- Interpretar briefing.
- Utilizar linguagem de programação JavaScript.
- Utilizar conceitos de programação orientada à objetos.
- Selecionar e interpretar requisitos de usuário.
- Utilizar boas práticas de programação e padrões de projetos.
- Utilizar termos técnicos nas rotinas de trabalho.

Problematização

Criar o *template* (tela) não deve oferecer um grande desafio, visto que o desenvolvedor já tem experiencia com o front-end. O maior desafio consiste então em de entender como realizar requisições a API e renderizar páginas HTML dinâmicas a partir da resposta da requisição.

Perguntas norteadoras:

- ✓ Como integrar o front-end com o back-end?
- ✓ Qual ferramenta pode facilitar a implementação dessas requisições?
- ✓ Como construir páginas HTML dinâmicas a partir de uma aplicação Node.js?
- ✓ O que é um HTML engine?
- ✓ Quais são as opções mais utilizadas no mercado?

Requisitos | Perfil Docente | Equipamentos

2. REQUISITOS DE ACESSO

Para matrícula no curso de Desenvolvedor Web Back-end: Node.js, o (a) candidato (a) deve ter no mínimo Ensino Fundamental completo.

Obs.: Para o bom desempenho no curso é importante que o aluno tenha conhecimentos básicos de lógica de programação, HTML, CSS e JavaScript.

9. PERFIL DO PESSOAL DOCENTE

O desenvolvimento da oferta ora proposta requer docentes com experiência profissional em projetos de website com HTML5, CSS 3, Bootstrap, JavaScript com Node.js e banco de dados e graduação em Tecnologia da Informação e/ou áreas afins.

8.1 Instalações e equipamentos específicos:

Laboratório de Software:

- Processador Intel® core i7 de 64 bits.
- RAM 8GB.
- Placa de vídeo 2GB.

Softwares:

- Node.js.
- Visual Studio Code.
- Navegadores web.

Introdução: Node.js

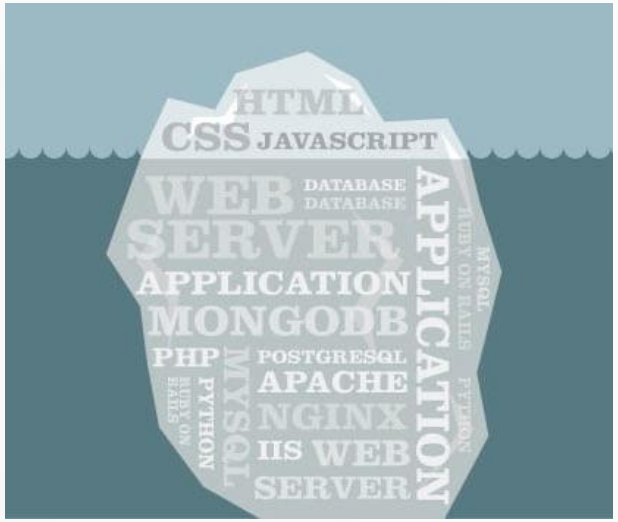
Front-End

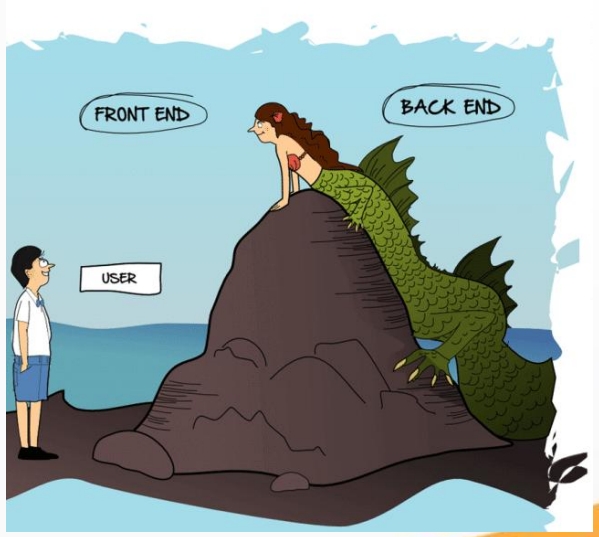
Vs.

Back-End

Front-End

Back-End





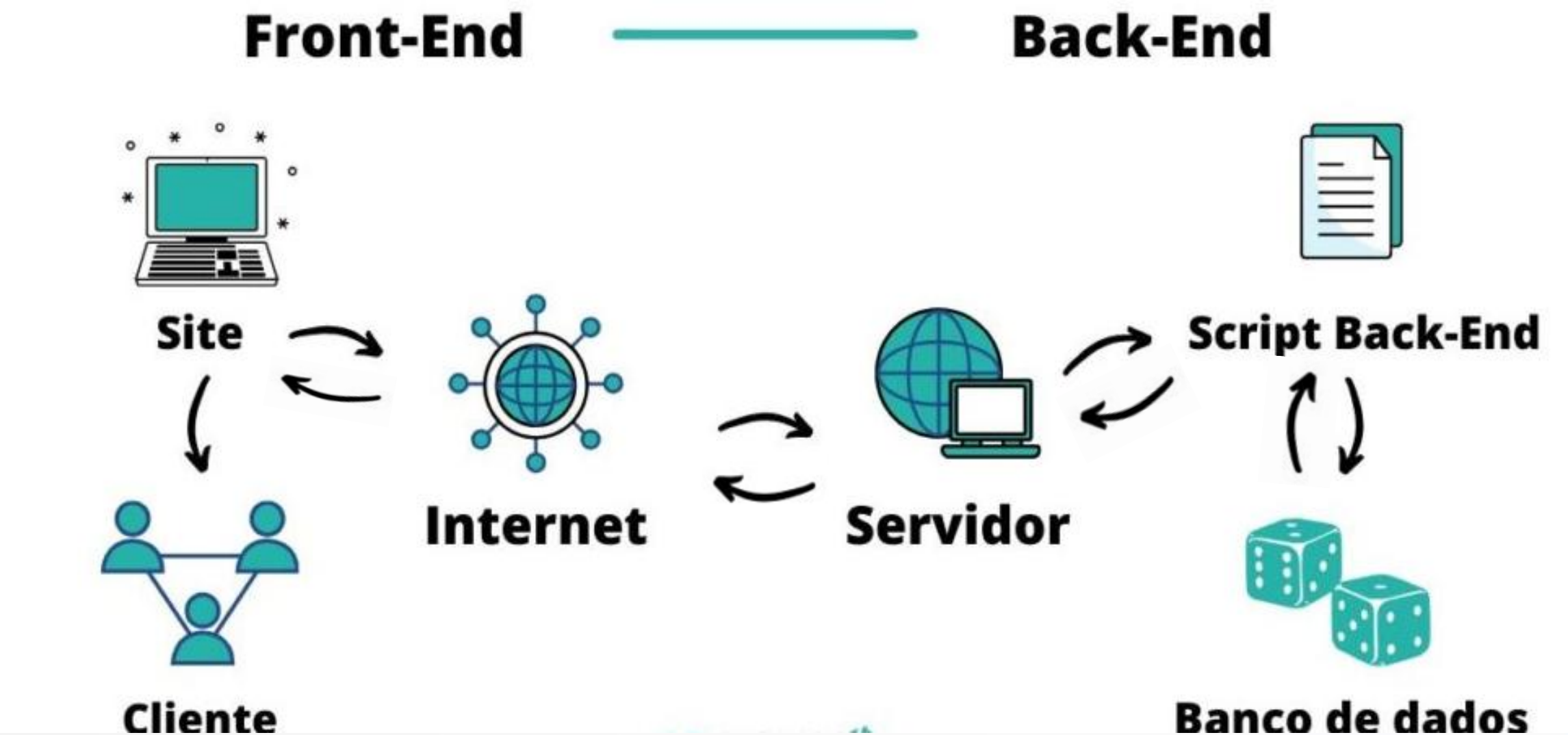
É necessário saber tudo?

Stacks – Back-End

Respectivos Frameworks, se necessário

Stacks – Front-End

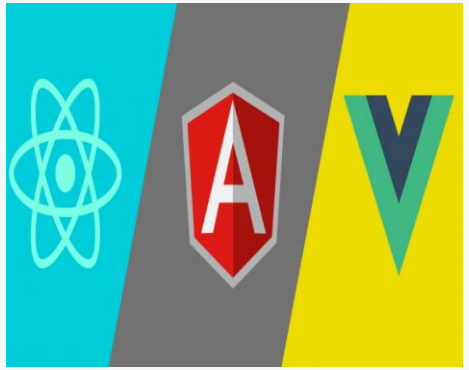
Stack – Banco de Dados


Front-End
Back-End
Site
Internet
Servidor
Script Back-End
Cliente
Banco de dados

Front-End

Linguagens

Frameworks



Frameworks Reativos

Back-End

Serviços HTTP

Linguagens

Frameworks

Banco de Dados

Na maioria dos casos é necessário **só uma STACK** para cada papel

A Stack do curso

O **Node.js** é um runtime de JavaScript, criado pelo desenvolvedor Ryan Dahl em 2009, possibilita o processamento e renderização do código **JavaScript** (linguagem de programação mais popular do mundo) no lado do servidor, ou seja, leva o **JavaScript** para o **back-end**.

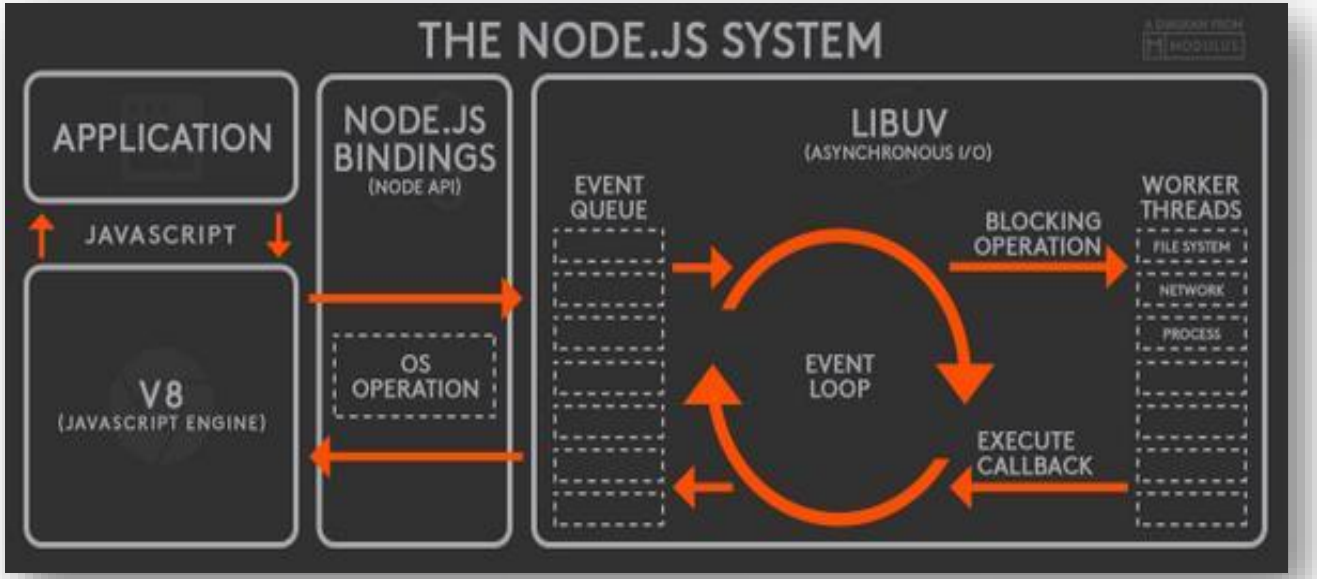
javaScript + Node.js

Uma das **stacks** mais poderosas para o back-end

Node.js

- O Node.js é um **ambiente de execução (runtime)** de JavaScript, criado por Ryan Dahl em 2009 e tem como suas principais funções: processar e executar códigos JavaScript no lado do servido.

- O Node.js já é uma tecnologia adotada o por grandes empresas no mercado e alguns dos principais motivos são: sua alta capacidade de escala, sua **arquitetura flexível** e de baixo custo.

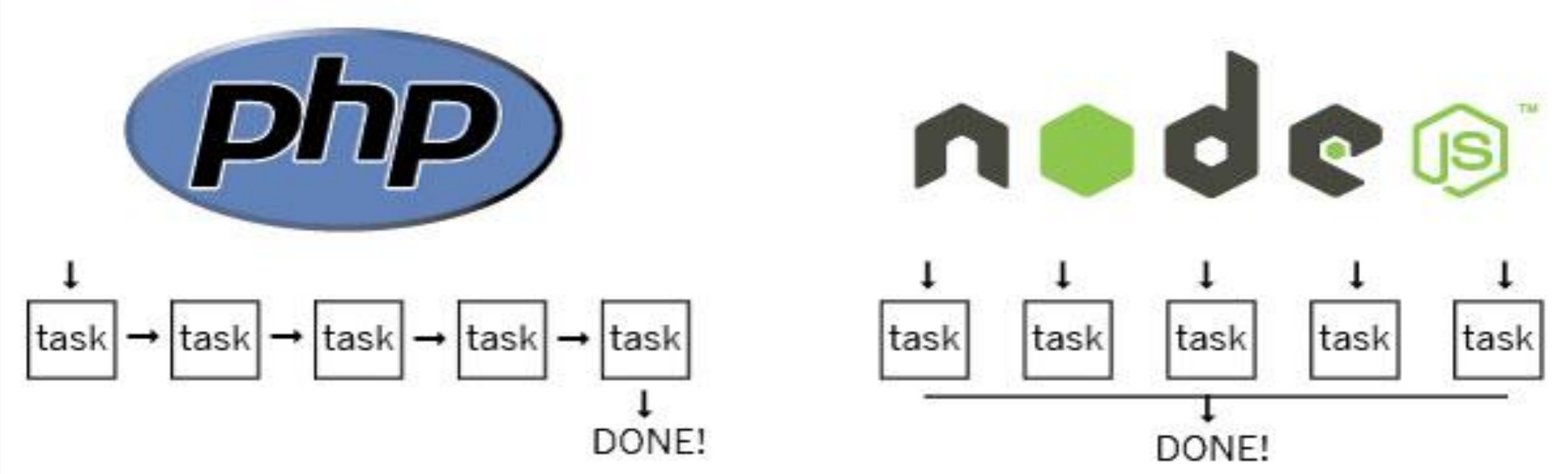


Fonte: https://stackoverflow.com/questions/42957176/nodejs-compilation-flow

A arquitetura do Node.js é constituída principalmente pelas bibliotecas V8(interpretador de javaScript) a biblioteca LIBUV(organiza os eventos e a execução de requisições).

Bloqueante vs Não Bloqueantes

- A execução do código é single thread, mas o node através da LIBUV lida com operações de E/S tais como leitura e gravação de disco, operações no banco de dados ou solicitações de rede de forma assíncrona. Isso torna a execução do código não bloqueante.

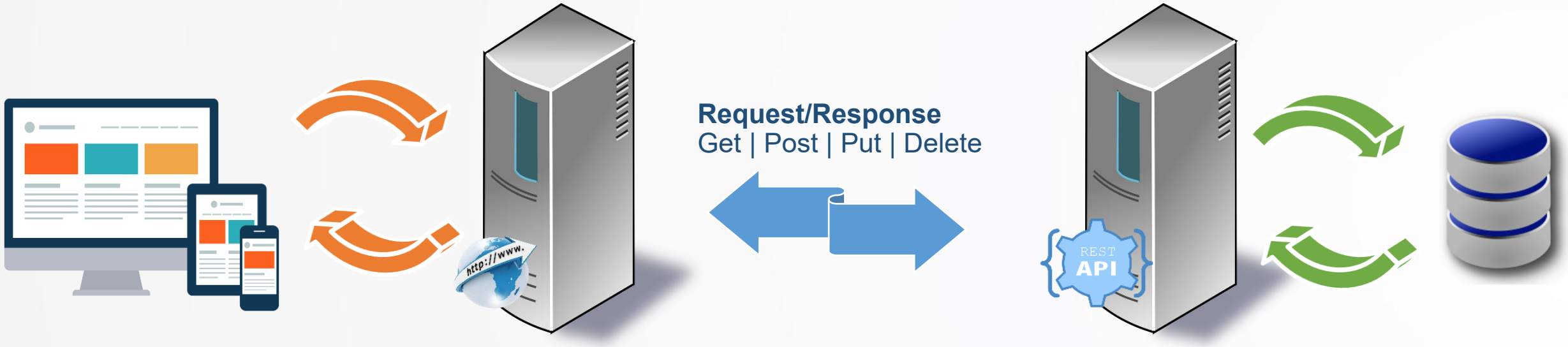


Fonte: https://shitech.com.pk/2017/02/10/php-vs-node-js-an-epic-battle-for-developer-mind-share/

Aplicação

Escopo

Sistema que armazene os dados de cadastro de pets, usuários que desejam doar ou adotar e facilite o processo de adoção animais em situação de abandono.

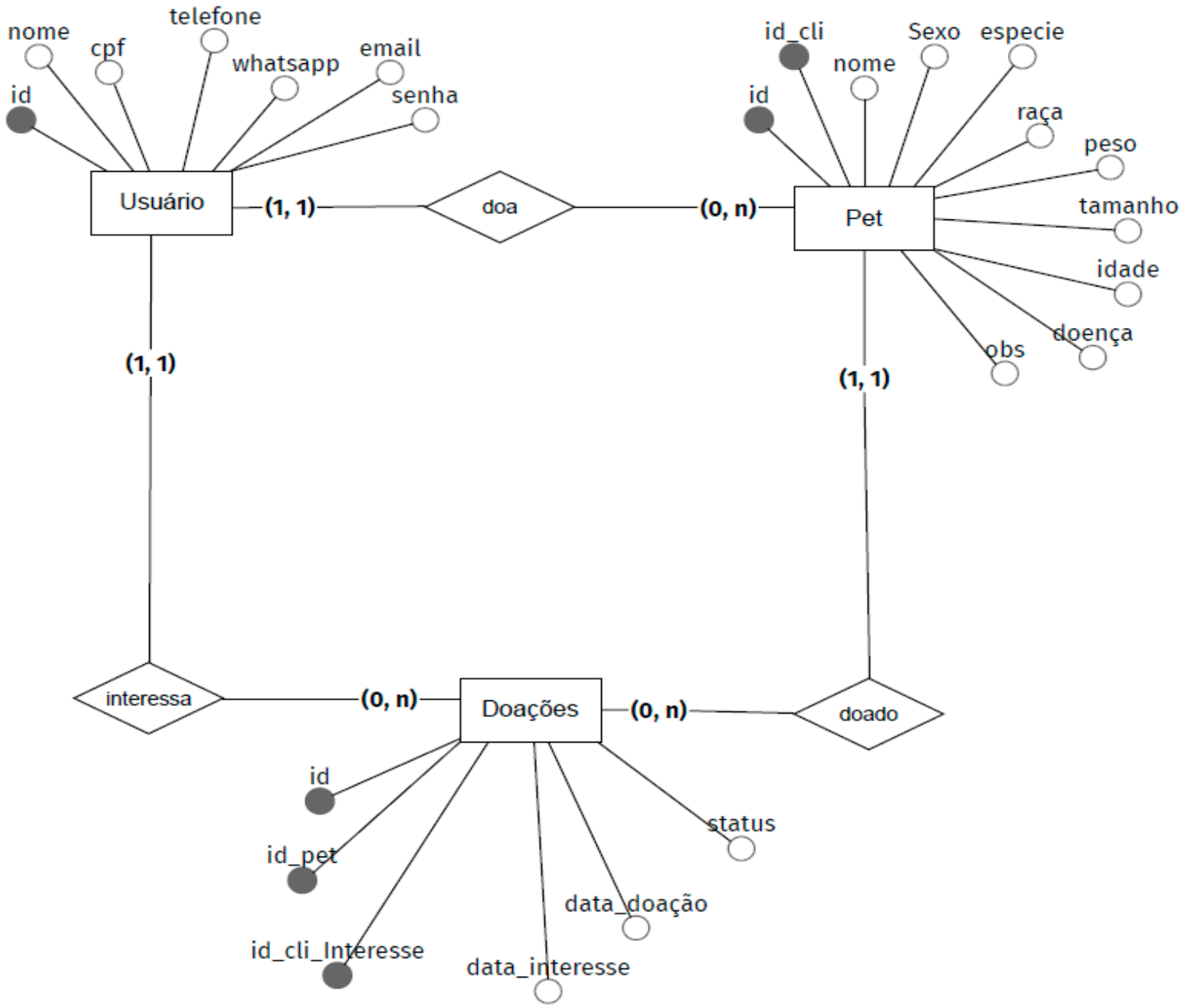


Request/Response
Get | Post | Put | Delete

Template Engine
HTTP: Porta 3000
Express/React

API REST-Ful
HTTP: Porta 3200
Express/mysql2
Nestjs/Prisma

MER


nome
cpf
telefone
whatsapp
email
senha
id
Usuário
(1, 1)
doa
(0, n)
Pet
id_cli
nome
Sexo
especie
id
raça
peso
tamanho
idade
doença
obs
(1, 1)
(1, 1)
interessa
(0, n)
Doações
(0, n)
doado
id
id_pet
id_cli_Interesse
data_interesse
data_doação
status